Department of Computer Science and Engineering
Assignment Questions
Course Code & Name : DSA01 - Object Oriented Programming with C++

Assignment Title:	Urban Complaint Management and Service Request System
Course Outcome(s) – CO:	**CO3:** Construct C++ programs using constructors, destructors, and operator overloading mechanisms to control object creation, destruction, and operator behaviour . **CO4:** Analyse and implement C++ applications using inheritance, types of inheritance, virtual base classes, abstract classes, pointers, and polymorphism to design reusable and extensible programs.
Bloom's Taxonomy Level	L3 – Apply; L4 – Analyse, L5-Evaluate
SDG Mapping (SDG 1-17)	SDG 9 – Industry, Innovation and Infrastructure; SDG 11 – Sustainable Cities and Communities; SDG 16 – Peace, Justice and Strong Institutions; SDG 12 – Responsible Consumption and Production

Problem Statement: Design, implement, and evaluate a menu-driven, object-oriented C++ application for an Urban Complaint Management and Service Request System to automate the registration, categorization, prioritization, assignment, status tracking, and resolution of civic complaints such as road damage, water supply, waste management, drainage, and streetlight issues. The system shall model citizens, complaints, departments, and service requests as encapsulated classes and manage multiple objects using suitable data types, control structures, arrays, and static members. Different complaint types shall be organized using inheritance and virtual functions to support runtime polymorphism and customized complaint processing. The application shall demonstrate default, parameterized, and copy constructors, destructors, pointers, and at least two meaningful operator overloads for comparing complaints and generating reports. A menu-driven interface shall provide complaint registration, department assignment, status updates, complaint comparison, resolution tracking, and service-performance report generation.

Assessment Rubrics

Criteria (CO Mapping)	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
Requirement Understanding, Data Types, Operators & Control Structures (CO1)	10	Correctly identifies the complaint-management workflow; uses suitable data types, operators, conditional statements and loops efficiently.	Identifies most requirements with minor gaps; uses data types and control structures appropriately	Basic requirements identified; some errors or inefficiencies in data types and control structures	Requirements poorly understood; frequent errors in data types, operators or control flow.

Class Design: Encapsulation, Access Specifiers & Member Functions (CO2)	15	Well-designed classes for proper encapsulation, access specifiers and cohesive member functions.	Classes are appropriately designed with minor issues in encapsulation or member functions.	Classes are present but encapsulation is weak or member functions are partially implemented.	Poor class design; inappropriate access specifiers and inadequate separation of data and behavior.
Static Members & Arrays of Objects (CO2)	10	Correctly uses static members for shared complaint statistics and arrays/collections of objects to manage multiple citizens, complaints and requests.	Static members and arrays of objects are implemented with minor logical errors.	Static members or arrays are partially implemented or underused.	Static members and arrays of objects are missing or non-functional.
Inheritance Hierarchy & Polymorphism (Virtual Functions) (CO4)	15	Clearly implements an inheritance hierarchy for different complaint/service types; virtual functions effectively provide runtime polymorphism for complaint processing and priority handling.	Inheritance and virtual functions are correctly implemented with limited polymorphic behaviour.	Basic inheritance is implemented, but overriding or polymorphism is incomplete.	No meaningful inheritance or polymorphism is implemented.
Constructors, Destructors & Operator Overloading (CO3)	15	Correctly implements default, parameterized and copy constructors; destructor performs appropriate resource cleanup; at least two meaningful operator overloads are correctly implemented for complaint comparison/report formatting.	Most constructors and at least one operator overload are correctly implemented with minor issues..	Some constructor types or one operator overload is implemented; destructor functionality is incomplete.	Constructors, destructor and operator overloading are missing, incorrect or cause runtime errors..
Menu-Driven Functionality, Correctness & Report Generation	25	All features—complaint registration, categorization, prioritization, department assignment, status updates, resolution	Most features work correctly with minor logical or formatting errors.	Core functions work, but some features such as prioritization, tracking or reporting are incomplete.	Program is unreliable; major features are missing or non-functional.

		tracking, comparison and service reports—work correctly through a clear menu-driven interface.			
Reflection: Design Decisions, SDG Relevance & Learning Outcomes	10	Thoughtful justification of design choices, clear connection to SDG 7/11/12 relevance, and honest, specific account of challenges faced and learning gained.	General justification of design choices; sustainability connection and challenges mentioned but lack depth.	Reflection present but generic; limited justification of design or vague account of learning outcomes.	No genuine reflection submitted, or content is generic with no real design/SDG/learning connection.

Deliverables

To receive full credit, each submission must include the following deliverables:

1. Pseudo code
2. Implementation & Results

3. Github Upload

1. PROBLEM STATEMENT

Assignment Title: Urban Complaint Management and Service Request System

Course Outcomes: CO3 – Construct C++ programs using constructors, destructors, and operator overloading mechanisms to control object creation, destruction, and operator behaviour. CO4 – Analyse and implement C++ applications using inheritance, types of inheritance, virtual base classes, abstract classes, pointers, and polymorphism to design reusable and extensible programs.

Design, implement, and evaluate a menu-driven, object-oriented C++ application for an Urban Complaint Management and Service Request System to automate the registration, categorization, prioritization, assignment, status tracking, and resolution of civic complaints such as road damage, water supply, waste management, drainage, and streetlight issues. The system shall model citizens, complaints, departments, and service requests as encapsulated classes and manage multiple objects using suitable data types, control structures, arrays, and static members. Different complaint types shall be organized using inheritance and virtual functions to support runtime polymorphism and customized complaint processing. The application shall demonstrate default, parameterized, and copy constructors, destructors, pointers, and at least two meaningful operator overloads for comparing complaints and generating reports. A menu-driven interface shall provide complaint registration, department assignment, status updates, complaint comparison, resolution tracking, and service-performance report generation.

SDG Mapping: SDG 9 – Industry, Innovation and Infrastructure; SDG 11 – Sustainable Cities and Communities; SDG 16 – Peace, Justice and Strong Institutions; SDG 12 – Responsible Consumption and Production.

2. PSEUDOCODE

Data structures

```
ENUM Priority { LOW=1, MEDIUM=2, HIGH=3, CRITICAL=4 }
ENUM Status   { REGISTERED, ASSIGNED, IN_PROGRESS, RESOLVED, CLOSED }
```

```
CLASS Citizen
    - name, contactNumber, address : string
    - static citizenCount : int
    + Citizen()                      // default
    + Citizen(name, contact, address)    // parameterized
    + Citizen(other)                 // copy
    + ~Citizen()
    + display()
```

```
CLASS Department
    - name : string, capacity, assignedCount, resolvedCount : int
    + receiveComplaint()
    + markResolved()
    + hasCapacity() : bool
```

```
CLASS Complaint  (ABSTRACT)
    - complaintId : int
    - citizen : Citizen
    - description, category : string
    - priority : Priority, status : Status
    - assignedDept : Department*          // pointer, non-owning
    - static totalComplaints, resolvedComplaints : int
    + Complaint()                        // default
    + Complaint(citizen, desc, priority, category)  // parameterized
    + Complaint(other)                   // copy
    + virtual ~Complaint()
    + ABSTRACT processComplaint()        // pure virtual -> polymorphism
    + virtual calculatePriorityScore() : int
    + virtual displayDetails()
    + assignDepartment(dept)
    + updateStatus(newStatus)
    + operator<(other) : bool            // OPERATOR OVERLOAD #1 (urgency compare)
    + FRIEND operator<<(ostream, Complaint) // OPERATOR OVERLOAD #2 (report line)
```

```
CLASS RoadDamageComplaint       EXTENDS Complaint  (overrides processComplaint, calculatePriorityScore)
```

```
CLASS WaterSupplyComplaint      EXTENDS Complaint  (overrides processComplaint, calculatePriorityScore)
CLASS WasteManagementComplaint  EXTENDS Complaint  (overrides processComplaint, calculatePriorityScore)
CLASS DrainageComplaint         EXTENDS Complaint  (overrides processComplaint, calculatePriorityScore)
CLASS StreetlightComplaint      EXTENDS Complaint  (overrides processComplaint, calculatePriorityScore)
```

```
CLASS ServiceRequest
   - requestId, complaintId : int
   - assignedDeptName, resolutionNotes : string
   - closed : bool
   - static requestCount : int
   + addResolutionNote(note)
   + closeRequest()
```

```
CLASS ComplaintManager
   - complaints : Complaint*[MAX]        // array of base-class pointers -> runtime polymorphism
   - departments : Department[5]
   - requests : ServiceRequest*[MAX]
```

Main algorithm

```
FUNCTION main():
   manager = new ComplaintManager()
   manager.initDepartments()             // pre-load 5 civic departments
   manager.run()
```

```
FUNCTION ComplaintManager.run():
   LOOP:
      PRINT menu (Register / Assign / Update / Compare / Track-Resolve /
                  Report / List / Exit)
      READ choice
      IF end-of-input: BREAK
      SWITCH choice:
         1 -> registerComplaint()
         2 -> assignToDepartment()
         3 -> updateComplaintStatus()
         4 -> compareComplaints()
         5 -> trackResolution()
         6 -> generateServiceReport()
         7 -> listComplaints()
         0 -> BREAK
         default -> PRINT "invalid"
```

```
FUNCTION registerComplaint():
    READ citizen name, contact, address
    citizen = new Citizen(name, contact, address)    // parameterized ctor
    READ category choice (1..5)
    READ description, priority
    SWITCH category:
        1 -> complaint = new RoadDamageComplaint(citizen, desc, priority)
        2 -> complaint = new WaterSupplyComplaint(citizen, desc, priority)
        3 -> complaint = new WasteManagementComplaint(citizen, desc, priority)
        4 -> complaint = new DrainageComplaint(citizen, desc, priority)
        5 -> complaint = new StreetlightComplaint(citizen, desc, priority)
    APPEND complaint TO complaints[]        // stored as base-class pointer
    complaint.processComplaint()            // virtual call -> runs the DERIVED version
    totalComplaints++                       // (handled inside Complaint ctor)
```

```
FUNCTION assignToDepartment():
    READ complaintId; FIND complaint
    READ department choice; dept = departments[choice]
    complaint.assignDepartment(dept)        // sets pointer, status = ASSIGNED
    request = new ServiceRequest(complaintId, dept.name)
    APPEND request TO requests[]
```

```
FUNCTION updateComplaintStatus():
    READ complaintId, newStatus
    FIND complaint; complaint.updateStatus(newStatus)
    IF newStatus is RESOLVED/CLOSED AND wasn't already:
        resolvedComplaints++
        dept.markResolved()
```

```
FUNCTION compareComplaints():
    READ id1, id2; FIND a, b
    IF a < b (operator<, compares calculatePriorityScore()):
        PRINT "a is lower priority than b"
    ELSE IF b < a:
        PRINT "a is higher priority than b"
    ELSE:
        PRINT "equal priority"
```

```
FUNCTION trackResolution():
    READ requestId; FIND request
    READ resolutionNote; request.addResolutionNote(note)
    READ closeFlag
    IF closeFlag:
        request.closeRequest()
```

```
    complaint = FIND complaint BY request.complaintId
    complaint.updateStatus(RESOLVED)
```

```
FUNCTION generateServiceReport():
  PRINT totalComplaints, resolvedComplaints, requestCount, citizenCount (static members)
  FOR each department: PRINT department.display()
  FOR each complaint: PRINT complaint using operator<<  // report generation
```

Priority-score weighting (drives polymorphic urgency)

```
baseScore = priority_enum_value * 10
RoadDamageComplaint.score       = baseScore + 5   // vehicle safety
WaterSupplyComplaint.score      = baseScore + 8   // health/hygiene critical
DrainageComplaint.score         = baseScore + 6   // flood risk
WasteManagementComplaint.score  = baseScore + 2
StreetlightComplaint.score      = baseScore + 1
```

3. Implementation — Source Code

CODE:

```
#include <iostream>
#include <string>
#include <limits>
using namespace
std; enum class
Priority {
  LOW = 1,

  MEDIUM = 2,

  HIGH = 3,

  CRITICAL = 4

};

enum class Status {
  REGISTERED,
  ASSIGNED,
  IN_PROGRESS,
  RESOLVED, CLOSED
};

string priorityToString(Priority
  p) { switch (p) {
    case Priority::LOW: return "LOW";

    case Priority::MEDIUM: return "MEDIUM";
    case Priority::HIGH: return "HIGH";
    case Priority::CRITICAL: return "CRITICAL";

  }

  return "UNKNOWN";

}

string statusToString(Status s)
  { switch (s) {
    case Status::REGISTERED: return "REGISTERED";
    case Status::ASSIGNED: return "ASSIGNED";

    case Status::IN_PROGRESS: return "IN_PROGRESS";
    case Status::RESOLVED: return "RESOLVED";
    case Status::CLOSED: return "CLOSED";

  }

  return "UNKNOWN";

}

class Citizen {
private:
  string name;

  string
  contactNumber;
  string address;
  static int citizenCount;
public:
  Citizen() {

    name = "Unknown";
    contactNumber = "N/A";
    address = "N/A";
```

```
    citizenCount++;
  }

  Citizen(string n, string contact, string
    addr) { name = n;
    contactNumber = contact;
    address = addr;
    citizenCount++;
  }

  Citizen(const Citizen& other) {
    name = other.name;
    contactNumber =
    other.contactNumber; address =
    other.address; citizenCount++;
  }

  ~Citizen() {
    citizenCount-
    -;
  }


  string getName() const {
    return name;
  }


  string getContactNumber() const
    { return contactNumber;
  }


  string getAddress()
    const { return
    address;
  }


  void display() const {

    cout << "Citizen: " << name

       << " | Contact: " << contactNumber

       << " | Address: " << address << endl;

  }


  static int getCitizenCount() {
    return citizenCount;
  }

};

int Citizen::citizenCount
= 0; class Department {
private:

  string name;

  int capacity;

  int
  assignedCount;
  int
  resolvedCount;
```

```
public:

  Department() {
    name =
    "General";
    capacity = 10;
    assignedCount = 0;

    resolvedCount = 0;

  }

  Department(string n, int cap) {
    name = n;
    capacity = cap;
    assignedCount =
    0;
    resolvedCount = 0;

  }

  Department(const Department& other) {
    name = other.name;
    capacity = other.capacity;
    assignedCount =
    other.assignedCount; resolvedCount
    = other.resolvedCount;
  }

  ~Department() {}

  string getName() const {
    return name;
  }

  int getAssignedCount() const {
    return assignedCount;
  }

  int getResolvedCount() const {
    return resolvedCount;

  }

  bool hasCapacity() const {

    return assignedCount < capacity;

  }

  void receiveComplaint()
    { if (hasCapacity()) {
      assignedCount++;

    }

  }

  void markResolved() {

    if (assignedCount >
      0) { assignedCount-
      -; resolvedCount++;
    }

  }

  void display() const {

    cout << "Department: " << name

       << " | Active: " << assignedCount
```

```
        << "/" << capacity
        << " | Resolved: " << resolvedCount << endl;
    }
};
class Complaint {
protected:
    int complaintId;
    Citizen citizen;
    string
    description;
    Priority
    priority; Status
    status;
    Department*
    assignedDept; string
    category;
    static int
    totalComplaints; static
    int resolvedComplaints;

public:
    Complaint()
      :
       complaintId(++totalComplaints
       ), citizen(),
       description("N/A"),
       priority(Priority::LOW),
       status(Status::REGISTERED),
       assignedDept(nullptr),
       category("General") {}
    Complaint(
      const Citizen&
      c, string desc,
      Priority p,
      string cat
    )
      :
       complaintId(++totalComplaints
       ), citizen(c),
       description(desc)
       , priority(p),
       status(Status::REGISTERED),
       assignedDept(nullptr),
       category(cat) {}
    Complaint(const Complaint& other)
      :
       complaintId(++totalComplaints
       ), citizen(other.citizen),
       description(other.description)
       , priority(other.priority),
       status(other.status),
```

```
    assignedDept(other.assignedDept),
    category(other.category) {}


 virtual ~Complaint() {}


 virtual void processComplaint() const = 0;


 virtual int calculatePriorityScore()
   const { return
   static_cast<int>(priority) * 10;
 }


 virtual void displayDetails() const {

   cout << "\nComplaint ID: #" << complaintId << endl;
   cout << "Category: " << category << endl;
   cout << "Description: " << description << endl;

   cout << "Priority: " << priorityToString(priority) <<
   endl; cout << "Status: " << statusToString(status) <<
   endl;
   cout << "Priority Score: " << calculatePriorityScore() << endl;
   citizen.display();


   if (assignedDept != nullptr)

     cout << "Assigned Department: "

        << assignedDept->getName() << endl;

   else

     cout << "Assigned Department: Unassigned" << endl;

 }


 void assignDepartment(Department* dept) {


   if (dept ==
     nullptr)
     return;
   if (!dept-
     >hasCapacity())
     {

     cout << "Department capacity is full.\n";
     return;
   }


   assignedDept = dept;

   dept->receiveComplaint();
   status = Status::ASSIGNED;
 }


void updateStatus(Status s)
                          {
```

```
  bool wasResolved =

    (status == Status::RESOLVED ||
     status == Status::CLOSED);


  status = s;


  bool isResolvedNow =

    (s == Status::RESOLVED ||
     s == Status::CLOSED);


  if (!wasResolved && isResolvedNow) {


     resolvedComplaints++;


    if (assignedDept != nullptr)
      assignedDept->markResolved();
  }

}

int getId() const { return complaintId;
}


Status getStatus() const
  { return status;
}


string getCategory() const {
  return category;
}


string getDescription() const {
  return description;
}


Priority getPriority() const {
  return priority;
}

bool operator<(const Complaint& other) const
  { return calculatePriorityScore()
     < other.calculatePriorityScore();

}

friend ostream& operator<<(
  ostream& os,
  const Complaint& c

) {

  os << "#" << c.complaintId

    << " [" << c.category << "] "

    << priorityToString(c.priority)
```

```
      << "/"
      << statusToString(c.status)
      << " - "
      << c.description;

    return os;
  }

  static int getTotalComplaints()
    { return totalComplaints;
  }

  static int
    getResolvedComplaints() {
    return resolvedComplaints;
  }
};

int Complaint::totalComplaints = 0;
int Complaint::resolvedComplaints = 0;
class RoadDamageComplaint : public Complaint {

public:

  RoadDamageComplaint(
    const Citizen& c,
    string desc,
    Priority p
  )
    : Complaint(c, desc, p, "Road Damage") {}

  void processComplaint() const override {
  cout << "-> Dispatching road inspection crew for complaint #"
       << complaintId << endl;
  }

  int calculatePriorityScore() const override
    { return
    Complaint::calculatePriorityScore() + 5;
  }
};
class WaterSupplyComplaint : public Complaint {

public:
```

```
  WaterSupplyComplaint(
    const Citizen& c,
    string desc,
    Priority p

  )

    : Complaint(c, desc, p, "Water Supply") {}


  void processComplaint() const override {

    cout << "-> Scheduling water utility technician for complaint #"

       << complaintId << endl;

  }


  int calculatePriorityScore() const override
    { return
    Complaint::calculatePriorityScore() + 8;
  }

};

class WasteManagementComplaint : public Complaint {


public:


  WasteManagementComplaint( const
    Citizen& c,
    string
    desc,
    Priority p
  )

    : Complaint(c, desc, p, "Waste Management") {}


  void processComplaint() const override {

    cout << "-> Routing waste-collection vehicle for complaint #"

       << complaintId << endl;

  }


  int calculatePriorityScore() const override
    { return
    Complaint::calculatePriorityScore() + 2;
  }

};


class DrainageComplaint : public Complaint {


public:


  DrainageComplaint(
```

```
    const Citizen&
    c, string desc,
    Priority p
  )

    : Complaint(c, desc, p, "Drainage") {}


  void processComplaint() const override {

    cout << "-> Sending drainage/flood-control team for complaint #"<< complaintId << endl;

  }


  int calculatePriorityScore() const override
    { return
    Complaint::calculatePriorityScore() + 6;
  }

};

class StreetlightComplaint : public Complaint {


public:


  StreetlightComplain
    t( const Citizen&
    c, string desc,
    Priority p
  )

    : Complaint(c, desc, p, "Streetlight") {}


  void processComplaint() const override {

    cout << "-> Logging streetlight repair ticket for complaint #"

       << complaintId << endl;

  }


  int calculatePriorityScore() const override
    { return
    Complaint::calculatePriorityScore() + 1;
  }

};

class ServiceRequest {
private:
  int requestId;
  int
  complaintId;
```

```
  string
  assignedDeptName;
  string resolutionNotes;
  bool closed;
  static int requestCount;


public:

  ServiceRequest() {

    requestId = ++requestCount;
    complaintId = -1;
    assignedDeptName = "Unassigned";
    resolutionNotes = "";
    closed = false;

  }


  ServiceRequest(
    int cid,
    string deptName

  ) {

    requestId = ++requestCount;
    complaintId = cid;
    assignedDeptName = deptName;
    resolutionNotes = "";
    closed = false;

  }


  ServiceRequest(const ServiceRequest& other)
    { requestId = ++requestCount;
    complaintId = other.complaintId;
    assignedDeptName = other.assignedDeptName;
    resolutionNotes = other.resolutionNotes;
    closed = other.closed;
  ~ServiceRequest() {}


  int getRequestId() const
    { return requestId;
  }


  int getComplaintId() const {
    return complaintId;
  }


  bool isClosed() const {
    return closed;
  }


  void addResolutionNote(string note) {


    if
      (!resolutionNotes.empty
      ()) resolutionNotes +=
```

```
      " | ";

    resolutionNotes += note;
  }

  void closeRequest()
    { closed = true;
  }

  void display() const {

    cout << "\nService Request #" << requestId
    << " | Complaint #" << complaintId
       << " | Department: "
       << assignedDeptName
       << " | Status: "
       << (closed ? "Closed" : "Open")
       << endl;

    if (!resolutionNotes.empty())
      cout << "Resolution Notes:
      "
         << resolutionNotes << endl;
  }

  static int getRequestCount() {
    return requestCount;
  }
};

int ServiceRequest::requestCount =
0; class ComplaintManager {

private:

  static const int MAX_COMPLAINTS = 100;
  static const int MAX_REQUESTS = 100;
  static const int MAX_DEPARTMENTS = 5;

  Complaint* complaints[MAX_COMPLAINTS]; int
  complaintCount;

  Department departments[MAX_DEPARTMENTS]; int
  departmentCount;
```

```
  ServiceRequest* requests[MAX_REQUESTS];
  int requestCount;

  Complaint* findComplaint(int id) {

    for (int i = 0; i < complaintCount; i++) {

      if (complaints[i]->getId() == id)
        return complaints[i];
    }

    return nullptr;
  }

public:

   ComplaintManager() {

    complaintCount = 0;
    departmentCount = 0;
    requestCount = 0;

    initDepartments();
  }

  ~ComplaintManager() {

    for (int i = 0; i < complaintCount;
      i++) delete complaints[i];for (int i
      = 0; i < requestCount; i++) delete
      requests[i];
  }

  void initDepartments() {

    departments[departmentCount++] =
      Department("Roads Department", 15);

    departments[departmentCount++] =
      Department("Water Department", 15);

    departments[departmentCount++] =
      Department("Waste Management Department", 15);

    departments[departmentCount++] =
      Department("Drainage Department", 10);
```

```
  departments[departmentCount++] =
    Department("Streetlight Department",
    10);
}

void registerComplaint() {


  if (complaintCount >= MAX_COMPLAINTS) {


    cout << "Complaint capacity reached.\n";
    return;
  }


  cin.ignore(
    numeric_limits<streamsize>::max(),
    '\n'
  );


  string name;
  string contact;
  string address;
  string
  description;


  cout << "\nEnter Citizen Name: ";
  getline(cin, name);


  cout << "Enter Contact Number: ";
  getline(cin, contact);


  cout << "Enter Address: ";
  getline(cin, address);


  Citizen
    citizen(
    name,
    contact,
    address
  );


  cout << "\nComplaint
  Category\n"; cout << "1. Road
  Damage\n";
  cout << "2. Water Supply\n";
  cout << "3. Waste Management\n";
  cout << "4. Drainage\n";
  cout << "5. Streetlight\n";


  cout << "Enter Choice: ";
```

```
int categoryChoice;

cin >> categoryChoice;


cin.ignore(
  numeric_limits<streamsize>::max(),
  '\n'
);


cout << "Enter Complaint Description: ";
getline(cin, description);


cout <<
"\nPriority\n"; cout
<< "1. LOW\n";  cout <<
"2. MEDIUM\n"; cout <<
"3. HIGH\n";
cout << "4. CRITICAL\n";


cout << "Enter Priority: ";


int priorityChoice;

cin >> priorityChoice;


if (priorityChoice < 1)
  priorityChoice = 1;


if (priorityChoice > 4)
  priorityChoice = 4;


Priority priority =
  static_cast<Priority>(priorityChoice
  );Complaint* complaint = nullptr;


switch (categoryChoice) {


  case 1:

    complaint =

      new RoadDamageComplaint(
        citizen,
        description,
        priority
      );

    break;


  case 2:

    complaint =
```

```
      new WaterSupplyComplaint(
        citizen,
        description,
        priority
      );

    break;


  case 3:

    complaint =

      new WasteManagementComplaint(
        citizen,
        description,
        priority
      );

    break;
  case 4:

    complaint =

      new DrainageComplaint(
        citizen,
        description,
        priority
      );

    break;


  case 5:

    complaint =

      new
        StreetlightComplaint(
        citizen,
        description,
        priority
      );

    break;


  default:


    cout << "Invalid category. "

       << "Using Road Damage.\n";


    complaint =

      new RoadDamageComplaint(
        citizen,
        description,
        priority
      );
}complaints[complaintCount++] = complaint;
```

```
   cout << "\n";


   // Runtime polymorphism
   complaint-
   >processComplaint();


   cout << "Complaint registered
   successfully.\n"; cout << "Complaint ID:
   #"
      << complaint->getId()

      << endl;

 }

void assignToDepartment() {


 if (complaintCount == 0) {


     cout << "No complaints available.\n";
     return;
   }


   cout << "\nEnter Complaint ID: ";


   int id;

   cin >> id;


   Complaint* complaint =
     findComplaint(id);


   if (complaint == nullptr) {
   cout << "Complaint not found.\n"; return;
   }


   cout << "\nDepartments\n";


   for (int i = 0;

      i <
      departmentCount;
      i++) {


    cout << i + 1

       << ". "

       << departments[i].getName()

       << endl;

   }
```

```
  cout << "Enter Department Choice: ";

  int choice;
  cin >> choice;

  if (choice < 1 ||
    choice > departmentCount) {

    cout << "Invalid department.\n";
    return;
  }

  Department* department =
    &departments[choice - 1];
  complaint->assignDepartment(
    department
  );

  ServiceRequest* request =
    new ServiceRequest(
      complaint->getId(),
      department->getName()
    );

  requests[requestCount++] = request;

  cout << "\nComplaint #"
     << id
     << " assigned to "
     << department->getName()
     << endl;

  cout << "Service Request #"
     << request->getRequestId()
     << " created.\n";
}
void updateComplaintStatus() {

  cout << "\nEnter Complaint ID: ";

  int id;
  cin >> id;
```

```
Complaint* complaint =
  findComplaint(id);if
  (complaint == nullptr) {


  cout << "Complaint not found.\n";
  return;
}


cout << "\nStatus\n";

cout << "1. ASSIGNED\n";
cout << "2. IN_PROGRESS\n";
cout << "3. RESOLVED\n";
cout << "4. CLOSED\n";


cout << "Enter Status: ";


int choice;

cin >> choice;


Status newStatus;


switch (choice) {


  case 1:

    newStatus = Status::ASSIGNED;
    break;


  case 2:

    newStatus = Status::IN_PROGRESS;
    break;


  case 3:newStatus = Status::RESOLVED; break;


  case 4:

    newStatus = Status::CLOSED;
    break;


  default:

    cout << "Invalid status.\n";
    return;
}


complaint->updateStatus(
  newStatus
```

```
  );

  cout << "Complaint #"
     << id
     << " status updated to "
     << statusToString(newStatus)
     << endl;
}

void compareComplaints() {

 if (complaintCount < 2) {

     cout << "At least two complaints "
      << "are required.\n";

     return;
  }

  cout << "\nEnter First Complaint ID: ";

  int id1;
  cin >> id1;

  cout << "Enter Second Complaint ID: ";

  int id2;
  cin >> id2;

  Complaint* first =
    findComplaint(id1);

  Complaint* second =
    findComplaint(id2);

  if (first == nullptr
    || second ==
    nullptr) {

    cout << "One or both complaints "
       << "were not found.\n";
```

```
    return;
  }
  if (*first < *second) {

    cout << "\nComplaint #"
       << id1
       << " has LOWER priority than Complaint #"
       << id2
       << endl;

  }
  else if (*second < *first) {

    cout << "\nComplaint #"
       << id1
       << " has HIGHER priority than Complaint #"
       << id2
       << endl;

  }
  else {

    cout << "\nBoth complaints have "
       << "equal priority.\n";
  }
}
void trackResolution() {

  if (requestCount == 0) {

    cout << "No service requests
    available.\n"; return;
  }

  cout << "\nEnter Service Request ID: ";

  int id;
  cin >> id;
```

```
ServiceRequest* request = nullptr;


for (int i = 0;

   i < requestCount;
   i++) {


  if (requests[i]->getRequestId() == id) {


     request = requests[i];
     break;
  }

}


if (request == nullptr) {


  cout << "Service request not
  found.\n"; return;
}


cin.ignore(
  numeric_limits<streamsize>::max(),
  '\n'
);


string note;


cout << "Enter Resolution Note: ";
getline(cin, note);


request->addResolutionNote(note);
cout << "Close request now?\n"; cout << "1. Yes\n";
cout << "0. No\n";


int closeChoice;

cin >> closeChoice;


if (closeChoice == 1) {


  request->closeRequest();


  Complaint* complaint =
    findComplaint(
      request->getComplaintId()

    );
```

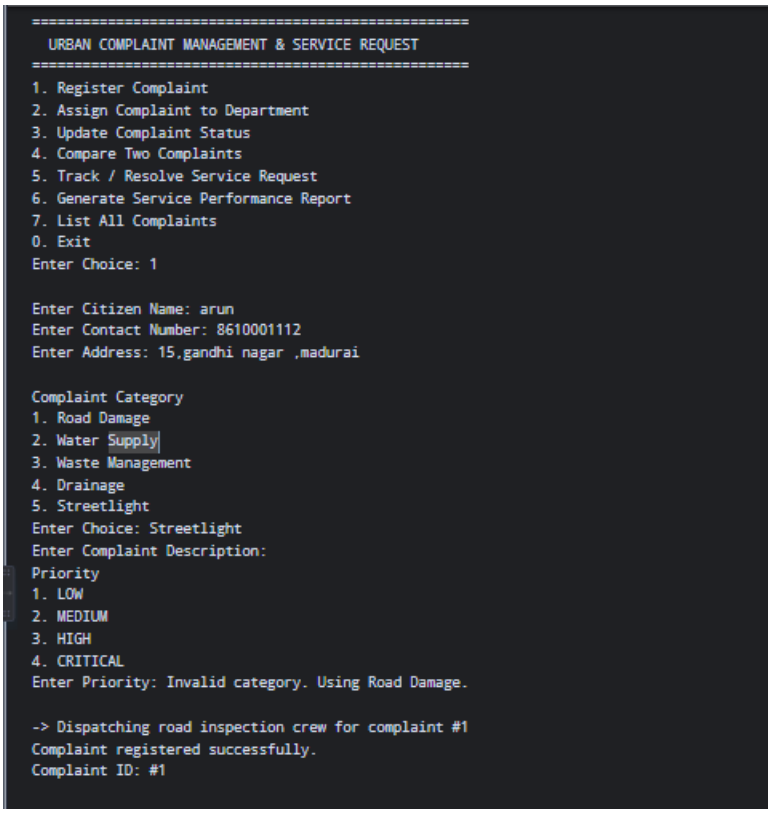
```
    if (complaint != nullptr) {


      complaint-
        >updateStatus(
        Status::RESOLVED
      );

    }


    cout << "Request closed successfully.\n";

  }


  request->display();

}


void generateServiceReport() const {
cout << "\n";

  cout << "=====================================================\n"; cout << "
               SERVICE PERFORMANCE REPORT\n";
  cout << "=====================================================\n";


  cout << "Total Complaints   : "

     << Complaint::getTotalComplaints()

     << endl;


  cout << "Resolved Complaints  : "

     << Complaint::getResolvedComplaints()

     << endl;


  cout << "Service Requests   : "

     << ServiceRequest::getRequestCount()

     << endl;


  cout << "Registered Citizens: "

     << Citizen::getCitizenCount()

     << endl;


  cout << "\nDepartment Summary\n";

  cout << "-----------------------------------------------------\n";


  for (int i = 0;
```

```
      i <
      departmentCount;
      i++) {


     departments[i].display();

   }
   cout << "\nComplaint Report\n";

   cout << "----------------------------------------------------\n";


   for (int i = 0;

      i < complaintCount;
      i++) {



     cout << *complaints[i] << endl;

   }


   cout << "====================================================\n";

 }




void listComplaints() const
                          {


  if (complaintCount == 0) {


     cout << "No complaints
     registered.\n"; return;
   }


   cout << "\n";

   cout << "================ ALL COMPLAINTS ================\n";


   for (int i = 0;

      i < complaintCount;
      i++)
      {complaints[i]-
      >displayDetails();


     cout << "----------------------------------------------------\n";

   }

 }
```

```
void run() {

  int choice;

   do {

     cout << "\n";

     cout << "=====================================================\n"; cout << "
     URBAN COMPLAINT MANAGEMENT & SERVICE REQUEST\n"; cout <<
     "=====================================================\n";

     cout << "1. Register Complaint\n";

     cout << "2. Assign Complaint to Department\n";
     cout << "3. Update Complaint Status\n";
     cout << "4. Compare Two Complaints\n";

     cout << "5. Track / Resolve Service Request\n";

     cout << "6. Generate Service Performance
     Report\n"; cout << "7. List All Complaints\n";
     cout << "0. Exit\n";

     cout << "Enter Choice:
     "; cin >> choice;

     switch (choice) {
       ase 1:

         registerComplaint();
         break;

       case 2:

         assignToDepartment();
         break;

       case 3:

         updateComplaintStatus();
         break;

       case 4:

         compareComplaints();
         break;

       case 5:

         trackResolution();
         break;

       case 6:

         generateServiceReport();
```

```
            break;

          case 7:
            listComplaints(
            ); break;

          case 0:
            cout << "\nThank you for using the
            system!\n"; break;default:
            cout << "Invalid choice. Try again.\n";
        }

      } while (choice != 0);
    }
};
int main() {

  ComplaintManager system;

  system.run();

  return 0;
}
```

4. RESULTS-PROGRAM OUTPUT:



5. REFLECTION: DESIGN DECISIONS, SDG RELEVANCE & LEARNING OUTCOMES:

Design decisions

a. Complaint is an abstract base class (processComplaint() is pure virtual) because every complaint category shares the same lifecycle (registered → assigned → in progress → resolved) but must customize how urgently it is treated and what dispatch action fires. Storing every complaint as a Complaint* in ComplaintManager lets the system treat all categories uniformly while still getting category-specific behaviour through virtual dispatch.

b. Department* inside Complaint is a non-owning pointer: a department can be linked to many complaints, so Complaint should never own or destroy it — ownership of Department objects stays with ComplaintManager. This is also why Complaint's destructor does not delete assignedDept.

c. ServiceRequest is kept separate from Complaint because the brief calls for service requests as their own concept: a request captures the administrative hand-off to a department (its own ID, resolution notes, closure state) independent of the complaint's own status field — similar to how real municipal ticketing systems separate a complaint record from a work-order record.

d. Priority scoring is virtual and weighted per category (e.g. water-supply complaints are weighted highest since outages affect health/hygiene; road-damage next for vehicle safety) so operator< produces a meaningful, domain-aware comparison rather than a raw enum comparison.

SDG RELEVANCE:

e. SDG 11 (Sustainable Cities and Communities): gives residents a structured channel to report infrastructure problems (roads, water, waste, drainage, streetlights).

f. SDG 9 (Industry, Innovation and Infrastructure): digitizes an otherwise manual civic complaint workflow into a trackable system.

g. SDG 16 (Peace, Justice and Strong Institutions): makes complaint status and department accountability visible and auditable through the service-performance report.

CHALLENGES AND LEARNING OUTCOMES:

h. Getting constructor/destructor counting right across copies (e.g. Citizen's static citizenCount incrementing in the copy constructor too, and decrementing in the destructor) required being deliberate about where objects are copied vs. referenced by pointer.

i. Using a virtual destructor in the abstract base class was essential — without it, deleting a RoadDamageComplaint through a Complaint* would only run the base destructor and leak/undefine derived-class cleanup.

j. Designing operator< around calculatePriorityScore() (itself virtual) meant the comparison automatically reflects each derived type's own weighting without compareComplaints() needing to know which concrete type it's comparing.

k. A natural next step would be replacing the fixed-size arrays with std::vector and adding file/database persistence so complaints survive across runs.